

STG-DGR: Fraud Detection on Streaming Transaction Graphs with Diffusion-based Generative Replay

Rui Ou

ourui@tongji.edu.cn

School of Computer Science and
Technology, Tongji University
Shanghai, China

Kun Zhu*

kzhu00@tongji.edu.cn

School of Computer Science and
Technology, Tongji University
Shanghai, China

Nana Zhang

nnzhang@dhru.edu.cn

School of Information and Intelligent
Science, Donghua University
Shanghai, China

Jiangtong Li

jiangtongli@tongji.edu.cn

School of Computer Science and
Technology, Tongji University
Shanghai, China

Chaochao Chen

zjuccc@zju.edu.cn

School of Computer Science and
Technology, Zhejiang University
Hangzhou, China

Changjun Jiang

cjjiang@tongji.edu.cn

School of Computer Science and
Technology, Tongji University
Shanghai, China

Abstract

Fraud detection on streaming transaction graphs (STGs) faces challenges on the catastrophic forgetting of previously learned fraud patterns when adapting to evolving patterns. Although some Graph Continual Learning (GCL) approaches mitigate this issue by storing and revisiting historical samples, practical storage constraints prevent them from fully preserving previous patterns. In this work, we propose STG-DGR, a streaming GNN model with diffusion-based generative replay that generates synthetic samples to retain previously learned patterns without storing real samples. The generation of replay samples for STGs faces two key challenges: (1) Heterogeneity challenge of generating STG samples with discrete adjacency table, user features, transaction features, and transaction timestamps. (2) Dependency challenge of capturing bottom-up dependencies across layers in STG samples. To address these challenges, STG-DGR integrates two novel components: (1) a Computational Subgraph Processor (CSP) that transforms heterogeneous STG samples into well-organized hierarchical subgraphs, and (2) a Diffusion-based Subgraph Generator (DSG) that captures the bottom-up dependencies using a novel Transformer-based Hierarchical Denoising Network (THDN), and generates synthetic replay samples that preserve these dependencies. Extensive experiments on four streaming fraud detection datasets demonstrate STG-DGR's superiority in reducing forgetting and improving accuracy over nineteen state-of-the-art baselines.

CCS Concepts

• Information systems → Data stream mining; • Security and privacy → Web application security.

*Corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License. *WWW '26, Dubai, United Arab Emirates*
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792195>

Keywords

Fraud Detection, Streaming Graph, Graph Neural Network, Continual Learning, Diffusion Model

ACM Reference Format:

Rui Ou, Kun Zhu, Nana Zhang, Jiangtong Li, Chaochao Chen, and Changjun Jiang. 2026. STG-DGR: Fraud Detection on Streaming Transaction Graphs with Diffusion-based Generative Replay. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792195>

1 Introduction

Detecting fraud in transaction data is crucial for maintaining financial security, preventing economic losses, and safeguarding trust in digital payment systems [8, 33, 51]. As illustrated in Figure 1, the massive stream of transaction data in real-world applications forms a large-scale **Streaming Transaction Graph (STG)** among users. To address the limitations of traditional fraud detection models in handling graph-structured data [1], recent research has increasingly adopted Graph Neural Networks (GNNs) to capture intricate relationships among users. However, most of them focus solely on static scenarios, overlooking the continuously evolving fraud patterns in the incoming data streams.

Graph Continual Learning (GCL) [28, 50] is a technique that enables models to continuously learn new patterns from a sequence of graph learning tasks. As shown in Figure 1, the transaction data can be periodically organized into incremental graphs (ΔG_1 , ΔG_2 , ΔG_3), and the fraud detection model is continuously updated based on these graphs. However, continual learning models are prone to overfitting new patterns, leading to **Catastrophic Forgetting** of previous fraud patterns [49]. **Replay-based method** is an effective approach to address this problem [20, 22], whose core idea is to update the model with the joint of new samples from current task and replay samples from previous ones. These replay samples are either sampled from previous tasks (**Sampling Replay methods, SRs**) [2, 26] or produced by a trainable replay generator (**Generative Replay methods, GRs**) [4, 36].

However, both replay methods have limitations when applied to STGs. For SRs, storing replay samples introduces significant memory overhead [26, 50]. As illustrated in Figure 2(a), for an L -layer

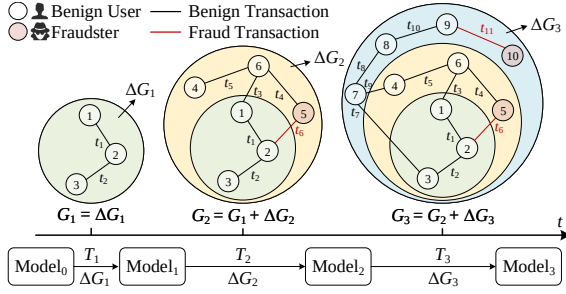
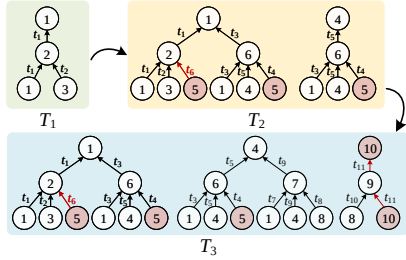
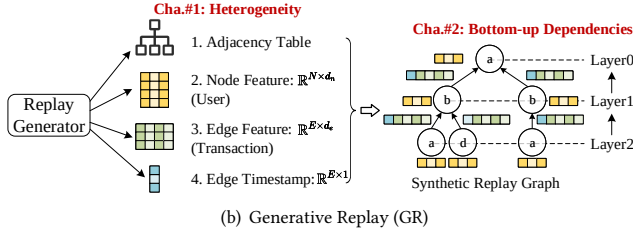


Figure 1: Illustration of a streaming transaction graph (STG).



(a) Sampling Replay (SR)



(b) Generative Replay (GR)

Figure 2: Illustration of SR and GR methods in STGs.

GNN model, SRs maintain a historical sample by storing the L -hop neighborhood around the target user, including all associated user features and transaction features. As the graph evolves and the GNN depth increases, maintaining complete historical patterns becomes increasingly impractical due to memory constraints. GRs provide a more memory-efficient alternative by training a replay generator to synthesize historical patterns. Although GRs achieve impressive success in the computer vision [4, 14, 25], their extension to STGs remains challenging due to two distinctive characteristics of STGs: **(1) Heterogeneity**. As illustrated on the left of Figure 2(b), generating a STG sample requires the generation of multiple heterogeneous data, including an adjacency table, node features, edge features, and edge timestamps. Here, N and E are the number of nodes and edges in the generated sample, while d_n and d_e denote the dimensions of node and edge features, respectively. **(2) Bottom-up Dependencies**. As illustrated on the right of Figure 2(b), a STG sample of the target user is hierarchically organized, with the target user as the root node. Bottom-up dependencies exist across layers, since GNNs typically compute node embeddings by aggregating neighbor information [5, 48]. Therefore,

the replay generator must capture these dependencies to produce high-quality synthetic STG samples.

Existing generative replay methods excel at generating single-type data but face difficulties with heterogeneous and interdependent graph data [4, 6]. To the best of our knowledge, SGNN-GR [36] is the only GR method designed for streaming graphs. It employs a GAN model to generate random walk sequences of a target node to approximate its heterogeneous neighborhood. However, the random walk approximation inevitably leads to significant information loss of node's neighborhood.

To address the aforementioned challenges, we propose a streaming GNN model with diffusion-based generative replay (**STG-DGR**) for fraud detection tasks on STGs. STG-DGR is the first study that incorporates the diffusion model to address the catastrophic forgetting problem in streaming fraud detection. Our main contributions are summarized as follows:

- To address the heterogeneity challenge for the generation of replay samples in STGs, STG-DGR is equipped with a **Computational Subgraph Processor (CSP)** module to process new samples into hierarchical subgraphs. The well-organized subgraphs not only allow the diffusion model to avoid manipulating the discrete adjacency table, but also encapsulate multidimensional neighborhood information around a central user.
- STG-DGR integrates a novel **Diffusion-based Subgraph Generator (DSG)** to generate synthetic samples, thereby preserving learned patterns in previous tasks without storing real samples. DSG extends the diffusion model to generate the subgraph around a user with a novel **Transformer-based Hierarchical Denoising Network (THDN)**.
- We conduct extensive experiments on four streaming fraud detection datasets. Experimental results demonstrate STG-DGR's superiority over nineteen state-of-the-art baselines, effectively reducing forgetting while improving accuracy.

2 Related Work

Fraud Detection with GNNs. GNN-based methods have attracted widespread attention in fraud detection tasks, such as web edit fraud [13, 24], review fraud [15, 38], transaction fraud [34, 40], etc. Most studies focus on static graphs [17, 52], neglecting the rich temporal information. Recent advancements in Temporal GNNs (TGNNs) have demonstrated promising results. RGTAN [39] learns the risk-aware representation in temporal graphs using a gated temporal attention network. TGFDN [45] proposes a group dynamics analyzer to enhance model's capability to identify group fraud on temporal graphs. CamFD [21] incorporates temporal contrastive learning and dynamic multivariate Gaussian distribution to identify fraudsters. DGNN-SR [46] proposes a multi-view time encoder, capturing temporal patterns based on both relative and absolute time. However, these studies still train models from the scratch and overlook streaming scenarios.

Graph Continual Learning (GCL) on Streaming Graphs. GCL is a widely used technique in streaming graph learning, aiming primarily at addressing the catastrophic forgetting problem [28, 49]. Replay-based methods represent a key branch of GCL, utilizing replay samples to preserve historical knowledge. Most existing replay studies adopt sampling replay (SR) methods, which store

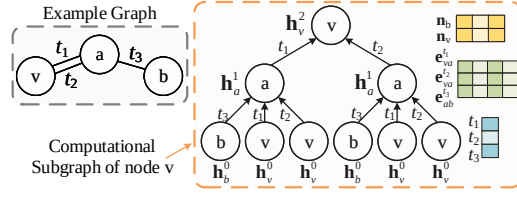


Figure 3: An example of computational subgraph.

historical samples in memory for replay when training new models [2, 19, 26, 50]. However, these methods introduce significant memory overhead to retain complete historical patterns. Generative replay (GR) methods typically train a generator to produce synthetic historical samples [25]. Although GR methods have shown success in computer vision, they cannot be directly applied to graph learning due to the heterogeneity and dependencies inherent in graph data. To our best knowledge, SGNN-GR [36] is the only GR methods in GCL fields. It designs a GAN-based model that generates random walk sequences to approximate the neighborhood of a target node. However, its approximation method cannot handle heterogeneous graphs and leads to a significant loss of neighbor information. Generative replay methods in graph continual learning still face many open challenges. To the best of our knowledge, this is the first work to introduce diffusion models to generate replay samples in streaming graph learning.

Diffusion Models. Diffusion models have demonstrated state-of-the-art generative performance in a wide range of tasks, including image synthesis [9, 10], text generation [42, 44], and audio generation [7, 43]. Despite their success in Euclidean domains, the application of diffusion models to graph data remains challenging due to the heterogeneity and dependency characteristics of graph data. Recent advances have attempted to extend diffusion models for graph generation tasks. DiGress [32] introduces a Markovian noise model for discrete denoising diffusion, which can preserve marginal structural information. GraphArm [12] designs a node-absorbing diffusion process which allows the diffusion model to directly operate on the discrete graph space. However, they typically focus on non-attributed graphs and are therefore unable to generate replay samples with node and edge features in Figure 2(b) for streaming graph learning.

3 Preliminaries

Streaming Transaction Graph (STG). An STG can be represented as the sequence of incremental graphs $\mathcal{STG} = \{\Delta G_i \mid i = 1, 2, \dots, T\}$, where T is the number of graphs (tasks). Each graph ΔG_i consists of a number of transaction records $\delta(t) = (u, v, \mathbf{n}_u, \mathbf{n}_v, \mathbf{e}_{uv}^t, t)$. $\delta(t)$ indicates user u trade with user v at time t , with node features $\mathbf{n}_u, \mathbf{n}_v \in \mathbb{R}^{d_n}$ and edge feature $\mathbf{e}_{uv}^t \in \mathbb{R}^{d_e}$.

Temporal Graph Neural Network (TGNN). TGNNs extend GNNs by incorporating temporal information, enhancing performance in streaming scenarios [24, 37, 41]. The representation of node v at layer l and time t , $\mathbf{h}_v^l(t)$ is computed as:

$$\mathbf{h}_v^l(t) = \text{COMBINE} \left(\mathbf{h}_v^{l-1}(t), \text{AGG} \left(\{(\mathbf{h}_u^{l-1}(t), \mathbf{e}_{uv}^t, t_{uv}) \mid u \in \mathcal{N}_v(t)\} \right) \right), \quad (1)$$

where $\mathcal{N}_v(t)$ is set of the neighbors for node v at time t . AGG and COMBINE are key operators in GNNs that aggregate neighbors and combine aggregated neighbor with center node, respectively. The node representation at layer 0 is the raw feature $\mathbf{h}_v^0 = \mathbf{n}_v$.

Computational Subgraphs. A computational subgraph for node v is a tree structure containing all the information needed to compute $\mathbf{h}_v^L$, where L is the number of GNN layers. As shown in Figure 3, with $L = 2$, $\mathbf{h}_v^2$ is computed bottom-up via Equation (1).

Fraud Detection on STGs. Given a streaming transaction graph $\mathcal{STG}$, where nodes represent users and edges denote transactions. Each edge has a label $y_{uv}^t \in \{0, 1, -1\}$ that indicates whether the user who initiates this transaction is fraudulent, benign, or unknown. Our goal is to classify all unknown transactions. Since a user may initiate multiple transactions over time, this task can be viewed as a streaming node classification problem.

4 Methodology

4.1 Overall Continual Learning Architecture

STG-DGR comprises three key modules: a **Computational Subgraph Processor (CSP)**, a **Diffusion-based Subgraph Generator (DSG)**, and a TGNN Classifier. CSP and DSG are two novel modules proposed in this work that enable subgraph-level generative replay for STG learning. For the TGNN classifier, we employ a 2-layer TGAT [41], which is a robust and widely-used backbone in temporal graph learning fields.

As shown in Figure 4, STG-DGR operates in four stages: (1) When new samples in ΔG_2 arrive, CSP processes the raw samples in ΔG_2 into well-organized hierarchical subgraphs, addressing heterogeneity problem; (2) DSG is trained with the processed samples by noising and denoising; (3) DSG generates replay samples by denoising from random graphs. A novel Transformer-based Hierarchical Denoising Network (THDN) parameterized the denoising process of DSG, which captures the bottom-up dependencies in the subgraphs; (4) TGNN is trained with the joint of new and replay samples, preserving previous patterns while learning new ones.

Next, we will introduce the two novel modules proposed in this paper, CSP and DSG, which address the two key challenges in graph generative replay, respectively.

4.2 Computational Subgraph Processor (CSP)

CSP addresses the heterogeneity challenge by processing the raw samples into well-organized hierarchical subgraphs. As shown in Figure 4, the raw computational subgraphs of new samples contain multiple heterogeneous data, including an adjacency table, user features, transaction features, and transaction timestamps. The adjacency table describes the dependency structure within the subgraph. However, diffusion models are typically designed to handle a single homogeneous matrix. Although some SOTA studies have attempted to modify the diffusion model so that it can directly operate the discrete adjacency table [12, 32], they cannot generate replay samples for STGs with node feature, edge feature, and edge timestamps. CSP proposes an alternative approach to transform raw computational subgraphs into multiple hierarchical feature matrices, allowing the diffusion model to avoid directly manipulating the discrete adjacency table. As shown in Figure 4, CSP processes the raw samples in both structural and feature levels.

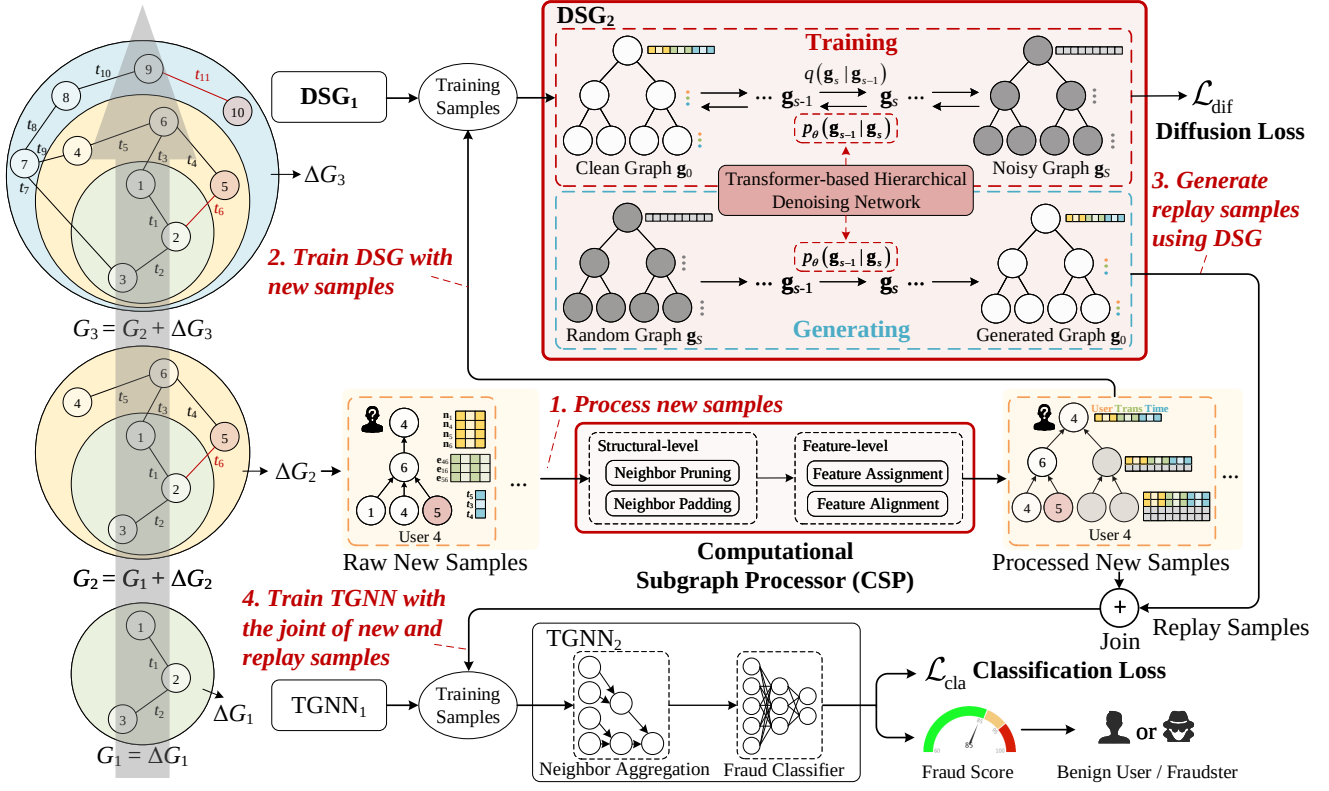


Figure 4: Overall architecture of STG-DGR.

Structural Level. Due to variations in the number of neighbors, the computational subgraphs of users typically exhibit a discrete tree-like structure, with inconsistent numbers of nodes and edges across different subgraph layers. To address this heterogeneity, CSP adopts neighbor pruning and padding inspired by TGAT [41] to ensure a fixed number of neighbors M . Figure 4 shows an example with $M = 2$. While TGAT primarily employs a similar approach for GPU acceleration, CSP utilizes it to address the heterogeneity challenge. Note that this approach standardizes the structure of computational subgraphs for each user in the STG, ensuring a consistent number of neighbors. This standardization establishes a unified dependency structure across all samples, enabling diffusion models to bypass the discrete adjacency table, which is no longer required to describe dependencies.

Feature Level. As shown in Figure 4, raw new samples exhibit dimensional heterogeneity among user features $\mathbb{R}^{N \times d_n}$, transaction features $\mathbb{R}^{E \times d_e}$, and transaction timestamps $\mathbb{R}^{E \times 1}$, both horizontally and vertically, which poses significant challenges for diffusion models to learn their joint distribution. Vertical heterogeneity stems from node features being node-specific, while edge features and timestamps are edge-specific. CSP addresses this by assigning all features to each subgraph layer and concatenating them horizontally. The horizontal heterogeneity is caused by the differences of feature dimension (d_n , d_e and 1). CSP employs a time encoder $\phi(\cdot)$ for timestamp encoding and a feed-forward network $f_e(\cdot)$ for edge feature transformation, mapping both to the same dimension as

node features (d_n):

$$\mathbf{te} = \phi(|\Delta t|) \in \mathbb{R}^{d_n}, \Delta t = t_{\text{parent}} - t_{\text{child}}; \quad (2)$$

$$\mathbf{e}' = f_e(\mathbf{e}) \in \mathbb{R}^{d_n}, \quad (3)$$

where $\mathbf{te}$ is the time embedding of child node calculated with the relative time span between parent and child node; $\mathbf{e}'$ is the transformed edge features. Features of the dummy nodes are filled with zeros.

In summary, CSP addresses the heterogeneity challenges by organizing heterogeneous raw features into hierarchical feature matrices (when $L = 2$, $\mathbf{m}^0 \in \mathbb{R}^{1 \times 3d_n}$, $\mathbf{m}^1 \in \mathbb{R}^{M \times 3d_n}$, and $\mathbf{m}^2 \in \mathbb{R}^{M^2 \times 3d_n}$). More importantly, CSP allows the diffusion model to focus on the hierarchical feature matrices without manipulating the discrete adjacency table because of the uniform dependency structures of processed samples.

4.3 Diffusion-based Subgraph Generator (DSG)

DSG addresses the dependency challenge and generates high-quality replay samples to mitigate catastrophic forgetting of previous learned benign/fraud patterns. As shown in Figure 4, the DSG workflow consists of two processes: *forward noising* and *reverse denoising*. Conventional diffusion models only learn the distribution of a single matrix, thus they simply add Gaussian noise to the matrix and denoise it [9]. In contrast, DSG requires careful design to learn the joint distribution of subgraphs produced by CSP during the

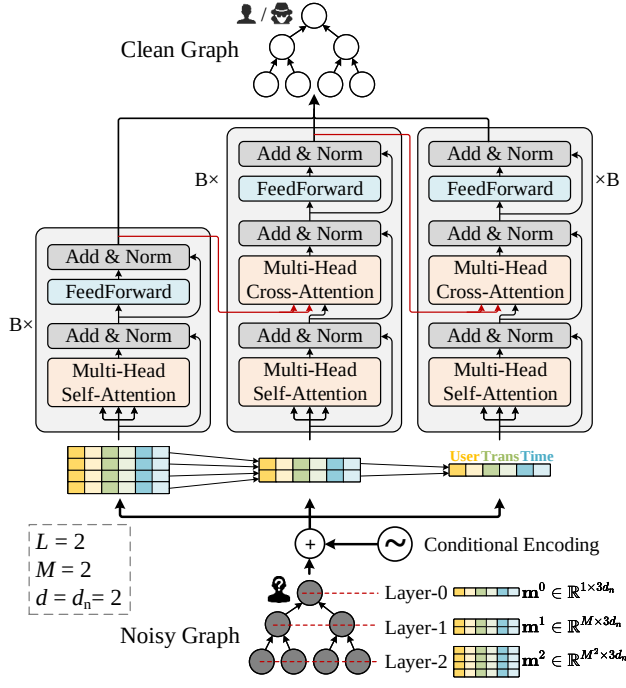


Figure 5: Transformer-based hierarchical denoising network.

two processes, which comprises three hierarchical feature matrices $\mathbf{g} = (\mathbf{m}^0, \mathbf{m}^1, \mathbf{m}^2)$ with bottom-up dependencies. The key challenge of DSG design is to capture the bottom-up dependencies among $\mathbf{m}^l$. Before delving into the details of the two processes, we first present a key proposition underlying the design of DSG.

PROPOSITION 1. *To capture the dependencies among $\mathbf{m}^0, \mathbf{m}^1, \mathbf{m}^2$, DSG can independently add noise to each matrix in the forward noising, and model the dependencies only in the reverse denoising.*

This proposition is theoretically supported by the evidence lower bound in diffusion models, which is justified in Appendix A. It allows the forward process to remain structurally simple and enables the backward phase to learn the joint distribution of inputs.

Forward noising process: According to Assumption 1, DSG adopts the factorized form of the forward noising kernel $q(\mathbf{g}_s | \mathbf{g}_{s-1})$ to simplify the forward process, adding Gaussian noise to the input matrices with variance schedule $\beta_1, \dots, \beta_S$:

$$q(\mathbf{g}_s | \mathbf{g}_{s-1}) = \prod_{l=1}^L q(\mathbf{m}_s^l | \mathbf{m}_{s-1}^l), \quad (4)$$

$$q(\mathbf{m}_s^l | \mathbf{m}_{s-1}^l) = \mathcal{N}(\mathbf{m}_s^l; \sqrt{1 - \beta_s} \mathbf{m}_{s-1}^l, \beta_s \mathbf{I}), \quad (5)$$

Reverse denoising process: Conventional diffusion models typically adopt UNets [23] as their denoising network, which perform well on images but cannot process interdependent matrices. To capture the bottom-up dependencies among $\mathbf{m}^l$ in the reverse denoising process, we propose a **Transformer-based Hierarchical Denoising Network (THDN)** as the denoising network of

Table 1: Dataset Statistics.

	S-FFSD	Credit	Wikipedia	Reddit
#Nodes	31,232	107,539	9,227	10,984
#Edges	77,881	5,125,108	157,474	672,447
#Edge Feat.	123	7	172	172
#Pos. Edge	5256(6.75%)	147,828(2.88%)	217(0.14%)	366(0.54%)
Time Span	90 days	180 days	30 days	30 days

DSG. As shown in Figure 5, THDN adopts a transformer-like architecture [30], comprising B stacked blocks that learn patterns for each matrix. Each block contains two key components: (1) a self-attention layer that captures intra-layer dependencies, and (2) a cross-attention layer that captures inter-layer dependencies (red lines in Figure 5):

$$\text{Self-Attn}(\mathbf{m}^l) = \text{Attn}(Q = \mathbf{m}^l, K = \mathbf{m}^l, V = \mathbf{m}^l), \quad (6)$$

$$\text{Cross-Attn}(\mathbf{m}^l) = \text{Attn}(Q = \mathbf{m}^l, K = \mathbf{m}^{l+1}, V = \mathbf{m}^{l+1}), \quad (7)$$

$$\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_{\text{model}}}}\right)V, \quad (8)$$

where $\mathbf{m}^l$ denotes the feature matrix of layer l ; $d_{\text{model}} = 3d_n$ indicates the dimension of the three input feature matrices, which is also the dimension of Q, K, V matrices in the attention mechanism [30]. For layer-2, we only employ the self-attention layer since it doesn't depend on other layers.

Conditional generation: We employ conditional encoding in the reverse processes of DSG to enable conditional generation of replay samples. As shown in the lower part of Figure 5, we leverage the positional encoding module in transformers [30] for conditional encoding. To enable denoising at arbitrary time step s , we implement a frequency-based encoding scheme using sine and cosine functions with learnable parameters to transform time step s into a conditional vector $CE_s \in \mathbb{R}^{d_{\text{model}}}$:

$$CE_s = \begin{cases} \sin\left(\frac{1}{10000^{2k/d_{\text{model}}}} \cdot s\right); i_{CE} = 2k, \\ \cos\left(\frac{1}{10000^{2k/d_{\text{model}}}} \cdot s\right); i_{CE} = 2k + 1, \end{cases} \quad (9)$$

where i_{CE} is the dimension index of CE_s . Besides, to address the class imbalance issue, DSG specifies the labels of generated samples through conditional encoding, increasing the proportion of fraud samples. Specifically, we encode the label y with a feed-forward network $CE_y = f_y(y) \in \mathbb{R}^{d_{\text{model}}}$. Finally, we add the conditional vectors to the input feature matrices, guiding the reverse denoising process of DSG $\mathbf{m}_{\text{cond}}^l = \mathbf{m}^l + CE_s + CE_y$.

In summary, DSG leverages a diffusion model to generate replay samples for STG learning, effectively mitigating catastrophic forgetting. It introduces an innovative design for the forward and reverse processes with respect to multiple input matrices, and addresses dependency challenges in replay sample generation for STGs through the novel THDN module in the reverse denoising process.

5 Experiments

5.1 Experimental Setup

Datasets. As shown in Table 1, we evaluate STG-DGR on four streaming datasets, including two transaction fraud dataset (S-FFSD [39] and Credit), as well as a web fraud dataset Wikepeida [27] and

Table 2: Overall performance on streaming fraud detection.

Datasets	S-FFSD		Credit		Wikipedia		Reddit	
Metrics	AUC↑	AFR↓	AUC↑	AFR↓	AUC↑	AFR↓	AUC↑	AFR↓
TGAT(Retrained)	74.24±2.41	/	90.35±1.24	/	83.69±0.70	/	65.56±0.70	/
RF	56.23±4.33	10.38±1.96	69.62±5.34	9.67±2.62	65.83±4.02	5.69±1.61	54.26±3.64	3.12±0.51
SVM	56.70±4.76	9.79±2.08	69.47±5.07	9.92±2.49	62.41±4.26	5.73±1.74	54.35±2.92	2.94±0.36
SemiGNN	63.98±2.89	6.91±1.76	81.35±2.53	8.94±1.57	75.01±2.83	5.28±1.91	58.65±3.16	2.75±0.39
CARE	64.20±2.70	6.82±1.60	81.96±2.61	8.37±0.97	74.56±3.27	5.61±1.82	59.24±2.60	2.65±0.43
TGAT	67.23±2.76	6.84±1.87	83.65±1.45	8.74±0.72	76.31±3.48	6.57±1.69	60.30±3.27	2.78±0.42
TADDY	67.41±2.60	6.70±1.92	83.18±1.57	8.83±1.84	77.52±3.83	5.26±1.62	60.84±3.09	2.71±0.51
MADG	67.18±2.81	6.83±1.76	84.35±1.75	8.26±1.53	76.85±2.29	5.85±1.75	61.72±3.95	2.56±0.47
GTAN	69.21±1.71	5.60±1.89	83.70±1.76	8.05±1.64	78.02±2.86	4.78±1.69	61.49±2.81	2.56±0.35
SAD	68.10±2.21	6.21±1.60	83.95±1.85	7.64±1.68	78.76±2.75	4.20±2.09	61.97±2.85	2.62±0.36
AdaDyGNN	67.20±2.46	6.46±1.73	83.87±1.97	8.01±1.76	78.01±2.89	4.72±1.66	61.64±3.06	2.83±0.41
RGTAN	69.40±2.38	5.47±1.68	84.24±1.60	7.80±1.62	78.23±3.09	4.57±1.87	61.87±2.70	2.87±0.30
ER-GNN	69.11±2.51	4.01±1.59	87.62±1.56	1.98±1.08	81.35±1.09	-0.58±0.86	62.24±1.57	-1.01±0.49
SSRM	69.03±2.36	4.26±1.75	87.53±1.28	1.56±0.78	81.83±0.75	-1.06±0.80	62.75±1.60	-1.32±0.41
SGNN-GR	69.31±2.28	4.21±1.39	87.86±1.34	1.49±0.97	81.47±1.21	-0.86±0.91	62.81±1.79	-1.39±0.52
DSLRL	69.38±2.25	4.34±1.27	87.70±1.46	1.52±0.80	81.62±1.03	-0.92±0.79	62.84±1.74	-1.37±0.49
ER-GNN+TGAT	69.97±2.26	3.65±1.28	87.98±1.49	1.76±0.69	82.14±1.32	-0.76±0.84	62.93±1.73	-1.26±0.54
SSRM+TGAT	69.85±1.92	3.73±1.32	88.06±1.50	1.48±0.74	82.31±1.07	-0.98±0.89	63.08±1.67	-1.30±0.59
SGNN-GR+TGAT	70.02±1.87	3.57±1.45	88.03±1.32	1.56±0.81	82.38±1.29	-0.80±0.76	62.98±1.81	-1.38±0.51
DSLRL+TGAT	70.15±1.76	3.61±1.20	88.08±1.46	1.51±0.77	82.40±1.22	-0.87±0.81	63.02±1.80	-1.36±0.61
STG-DGR	72.76±1.03	2.31±0.65	89.84±0.98	1.18±0.51	84.48±0.81	-1.27±0.74	64.95±1.62	-1.56±0.48

a posting fraud dataset Reddit [27]. All datasets are divided into 10 tasks in chronological order for continual learning.

Baselines. We compare STG-DGR with two typical machine learning methods in fraud detection fields, random forest (RF) and support vector machine (SVM), as well as seventeen state-of-the-art graph learning baselines for continual learning tasks, including two static GNN methods (SemiGNN [35] and CARE [3]), seven temporal GNN methods (TGAT [41], TADDY [18], MADG [47], GTAN [40], SAD [27], AdaDyGNN [16], and RGTAN [39]), four static GCL methods (ER-GNN [50], SSRM [26], SGNN-GR [36], and DSLR [2]), and four temporal GCL methods. The four temporal GCL baselines employ the same TGNN backbone (i.e., TGAT) as STG-DGR to ensure a fair comparison.

Implementation Details. STG-DGR employs a 2-layer TGAT as its backbone classifier with the following hyperparameters across all datasets. CSP sets $M = 20$ neighbors and an embedding dimension of $d = d_n = 128$. DSG follows the variance schedule $\beta_1, \dots, \beta_S$ as used in DDPM [9], with $S = 500$ diffusion steps and $B = 6$ attention blocks. The number of generated replay samples are set to $\gamma = 10\%$ of the new samples. For the S-FFSD dataset, we use a 60%-20%-20% split for training, validation, and test sets [39], while for the Credit, Wikipedia, and Reddit datasets, we use a 70%-15%-15% split [16, 27]. All experiments are conducted on a server with a NVIDIA V100 (32GB), Intel Xeon Gold 6230 CPUs and 64GB RAM.

Metrics. We evaluate models using AUC (area under ROC curve) and AFR (average forgetting rate). AUC is a threshold-independent metric and is suitable for fraud detection tasks with class imbalance issues. AFR quantifies the general levels of forgetting in streaming

learning tasks [50]:

$$AFR = \frac{1}{T} \sum_i^T p_{i,i} - p_{T,i}, \quad (10)$$

where $p_{i,j}$ denotes the model performance (AUC) on task j after training on the task sequence from 1 to i . A lower AFR indicates better retention of previously learned patterns.

5.2 Evaluation on Streaming Fraud Detection

As shown in Table 2, we evaluate STG-DGR against nineteen baselines in streaming fraud detection tasks. To ensure fair comparison, all models are incrementally trained with a stream composed of ten chronologically fraud detection tasks.

The first row of Table 2 reports the AUC of the fully retrained TGAT model. Although it generally achieves better performance compared to incrementally trained models, it requires approximately ten times more computational resources in terms of both time and space. The baselines can be categorized into five groups: (1) **Traditional machine learning models (RF and SVM)** are effective for fraud detection on tabular data but limited in capturing graph relationships. (2) **Static GNN models (SemiGNN and CARE)** learn the fraud patterns on graphs with GNNs but ignore the rich temporal information in streaming graphs. (3) **Temporal GNN models (TGAT, TADDY, MADG, GTAN, SAD, AdaDyGNN, and RGTAN)** represent the state-of-the-art models in temporal graph learning. They incorporate advanced temporal modules to effectively capture temporal dynamics, demonstrating consistently superior performance across all datasets.

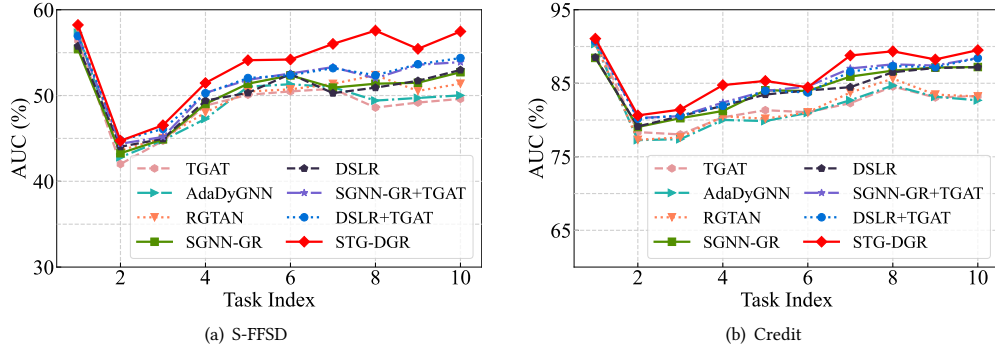


Figure 6: Long-term forgetting: AUC of the first task throughout the continual learning process.

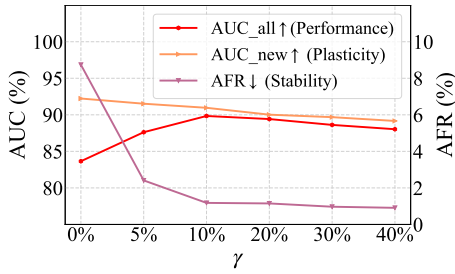


Figure 7: Stability and plasticity trade-off in Credit.

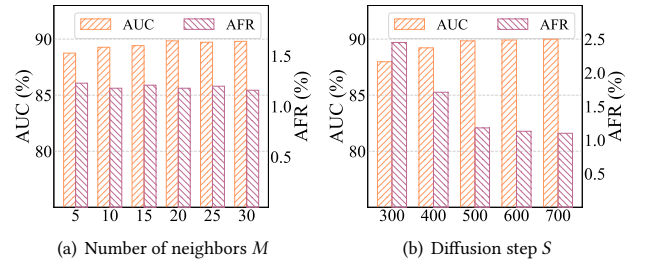


Figure 8: Impact of hyper-parameters M and S .

However, the aforementioned models fail to address the catastrophic forgetting problem, resulting in a high AFR and significant AUC degradation compared to the retrained models. For example, the incrementally trained TGAT exhibits a performance decline of 5.26% to 7.38% compared to its retrained version. **(4) Static graph continual learning (GCL) models (ER-GNN, SSRM, SGNN-GR, and DSLR)** mitigate forgetting through historical sample replay and loss regularization techniques. Although they employ static GNNs [11, 31] that cannot capture temporal information, the lower AFRs enable them to outperform four temporal GNN models. **(5) Temporal GCL models (ER-GNN+TGAT, SSRM+TGAT, SGNN-GR+TGAT, and DSLR+TGAT).** We further enhance static GCL baselines by replacing their backbone GNN with TGAT, the same TGNN backbone used in STG-DGR. By incorporating temporal information, these models demonstrate better performance compared to their static counterparts.

STG-DGR achieves state-of-the-art performance in both AUC and AFR across all datasets, outperforming nineteen incrementally trained baselines and performing comparably to retrained TGAT. This superior performance stems from two key innovations. First, STG-DGR organizes structural, temporal, and feature information in computational subgraphs with the CSP module. Second, STG-DGR learns the subgraphs with a novel diffusion-based subgraph generator (DSG), generating high-quality replay samples.

5.3 Evaluation on Long-term Forgetting

To offer additional insight into our approach, we conduct a case study to demonstrate how the models' performance of the first task

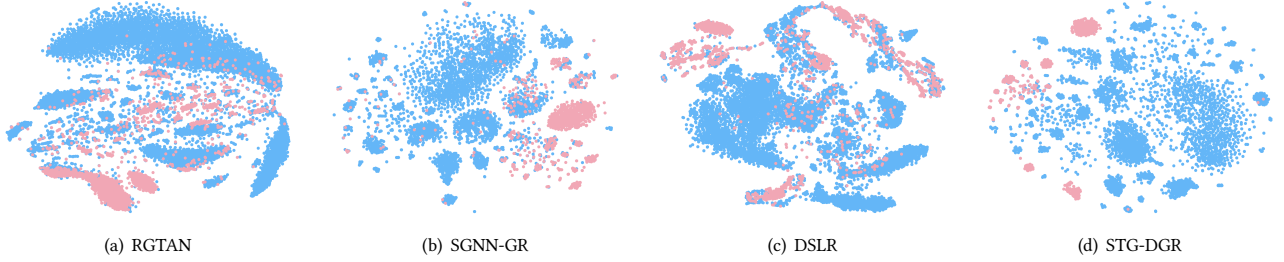
evolves during continual learning process across two transaction fraud datasets. As illustrated in Figure 6, for all test cases, the AUC of first task exhibits a significant decline as the second task arrives, mainly due to the substantial differences in data distribution between tasks. Although model performance gradually recovers with the arrival subsequent tasks, non-continual learning approaches (TGAT, AdaDyGNN, RGTAN) still demonstrate significant performance declines compared to their initial levels. In contrast, STG-DGR successfully maintains performance stability, eventually recovering to levels comparable to its initial performance. This robust performance preservation demonstrates STG-DGR's effectiveness in mitigating catastrophic forgetting through its innovative diffusion-based generative replay mechanism.

5.4 Stability and Plasticity Trade-off

The stability and plasticity trade-off is a fundamental problem in continual learning. In this work, stability of the fraud detection model refers to its capability to retain previously learned benign and fraudulent patterns, while plasticity refers to its capability to learn new patterns. As a generative replay method, the ratio of replay samples γ in STG-DGR is a key parameter to control this trade-off. We evaluate stability using AFR, plasticity using AUC of the new task, and overall performance using the average AUC of all tasks. As shown in Figure 7, increasing γ leads to a decrease in AFR, indicating improved stability, since more historical user patterns are revisited through generative replay. In contrast, the AUC of new task decreases with higher γ , reflecting reduced plasticity

Table 3: Ablation study.

Datasets	S-FFSD		Credit		Wikipedia		Reddit	
Metrics	AUC↑	AFR↓	AUC↑	AFR↓	AUC↑	AFR↓	AUC↑	AFR↓
TGAT	67.23±2.76	6.84±1.87	83.65±1.45	8.74±0.72	76.31±3.48	6.57±1.69	60.30±3.27	2.78±0.42
w/o Time	70.17±1.68	3.83±1.56	87.97±1.53	1.79±0.80	82.74±2.35	-0.54±0.83	62.96±2.30	-0.96±0.62
Diffusion→GAN	70.79±1.79	3.28±1.28	88.03±1.32	1.56±0.81	82.38±1.29	-0.80±0.76	62.98±1.81	-1.38±0.51
w/o FeatAlign	71.94±1.26	2.84±0.89	89.24±1.32	1.47±0.69	83.75±1.63	-0.84±0.68	63.31±1.80	-1.27±0.57
w/o CrossAttn	71.23±1.41	3.07±1.05	88.35±1.03	1.39±0.78	83.24±2.16	-0.65±0.92	63.25±1.97	-1.19±0.63
STG-DGR	72.76±1.03	2.31±0.65	89.84±0.98	1.18±0.51	84.48±0.81	-1.27±0.74	64.95±1.62	-1.56±0.48

**Figure 9: Visualization results on Credit dataset, where blue and pink nodes represent benign users and fraudsters, respectively.**

due to the diminished focus on new data. Both stability and plasticity contribute to the model’s overall performance (i.e., the average AUC of all tasks), and we empirically set $\gamma = 10\%$ to balance them.

5.5 Ablation Study

To demonstrate the contribution of different components and mechanisms in STG-DGR, we conduct an ablation study on the following variants: (1) **w/o Time** is the static version of proposed STG-DGR that ignores all temporal information on streaming graphs. (2) **Diffusion→GAN** replaces the diffusion model with a GAN-based replay generator used in SGNN-GR [36]. (3) **w/o FeatAlign** disables the feature-level components in CSP. We can not disable the whole CSP module because the structure-level components fix the structure of computational subgraphs which is necessary for the latter DSG generation. (4) **w/o CrossAttn** disables the cross-attention layers in THDN as shown in Figure 5. In this case, STG-DGR can not capture the inter-layer dependency in subgraphs since the feature matrix of each layer will be denoised independently. The ablation study results in Table 3 demonstrate that each disabled component contributes significantly to STG-DGR’s overall performance.

5.6 Hyperparameter Impact and Visualization

We analyze the impact of two key hyperparameters in STG-DGR: the number of neighbors in CSP M and the diffusion step in DSG S . As shown in Figure 8(a) a larger M allows STG-DGR to capture more neighbor information, improving its performance. However, it also increases computational cost during training. To balance performance and efficiency, we set $M = 20$. Figure 8(b) illustrates the impact of the diffusion step S . In general, a larger number of

diffusion steps enables a more fine-grained denoising process for DSG, potentially enhancing the quality of generated samples. However, increasing the number of steps also incurs higher training costs. Experimental results show that beyond 500 steps, the performance gains become marginal. Therefore, we empirically set $S = 500$.

We also apply t-SNE [29] to reduce the dimensionality of the node embeddings, visualizing the fraud detection performance of STG-DGR. As illustrated in Figure 9, compared to three representative baselines, STG-DGR demonstrates better capability to distinguish fraudsters from benign users.

6 Conclusion

In this work, we propose STG-DGR, a novel fraud detection model for streaming transaction graphs (STGs) that addresses the catastrophic forgetting problem of previous fraud patterns with diffusion-based generative replay. STG-DGR employs a Computational Subgraph Processor (CSP) and a Diffusion-based Subgraph Generator (DSG) to tackle two key challenges for generating replay samples on STG learning tasks: heterogeneity and bottom-up dependencies. Experimental results demonstrate the effectiveness of STG-DGR compared to nineteen state-of-the-art baselines across four fraud detection datasets.

Acknowledgments

This work is supported in part by the National Key Research and Development Program of China under Grant YS2022YFB4500205 and in part by the National Natural Science Foundation of China under Grant 62302337, 62402098, 62402341.

References

- [1] Fidelis Obukohwo Aghware, Arnold Adimabua Ojugo, Wilfred Adigwe, Christopher Chukwufumaya Odiakaoe, Emma Obiajulu Ojei, Nwanze Chukwudi Ashioba, Margareth Dumebi Okpor, and Victor Ochuko Geteloma. 2024. Enhancing the random forest model via synthetic minority oversampling technique for credit-card fraud detection. *Journal of Computing Theories and Applications* 1, 4 (2024), 407–420.
- [2] Seungyeon Choi, Wonjoong Kim, Sungwon Kim, Yeonjun In, Sein Kim, and Chanyoung Park. 2024. DSLR: Diversity enhancement and structure learning for rehearsal-based graph continual learning. In *Proceedings of the ACM Web Conference 2024*. 733–744.
- [3] Yingdong Dou, Zhiwei Liu, Li Sun, Yutong Deng, Hao Peng, and Philip S Yu. 2020. Enhancing graph neural network-based fraud detectors against camouflaged fraudsters. In *Proceedings of the 29th ACM international conference on information & knowledge management*. 315–324.
- [4] Rui Gao and Weiwei Liu. 2023. Ddgr: Continual learning with deep diffusion-based generative replay. In *International Conference on Machine Learning*. PMLR, 10744–10763.
- [5] Justin Gilmer, Samuel S Schoenholz, Patrick F Riley, Oriol Vinyals, and George E Dahl. 2020. Message passing neural networks. *Machine learning meets quantum physics* (2020), 199–214.
- [6] Gabriele Graffieti, Davide Maltoni, Lorenzo Pellegrini, and Vincenzo Lomonaco. 2023. Generative negative replay for continual learning. *Neural Networks* 162 (2023), 369–383.
- [7] Zhifang Guo, Jianguo Mao, Rui Tao, Long Yan, Kazushige Ouchi, Hong Liu, and Xiangdong Wang. 2024. Audio generation with multiple conditional diffusion model. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 18153–18161.
- [8] Waleed Hilal, S Andrew Gadsden, and John Yawney. 2022. Financial fraud: a review of anomaly detection techniques and recent advances. *Expert systems With applications* 193 (2022), 116429.
- [9] Jonathan Ho, Ajay Jain, and Pieter Abbeel. 2020. Denoising diffusion probabilistic models. *Advances in neural information processing systems* 33 (2020), 6840–6851.
- [10] Yi Huang, Jiancheng Huang, Yifan Liu, Mingfu Yan, Jiaxi Lv, Jianzhuang Liu, Wei Xiong, He Zhang, Liangliang Cao, and Shifeng Chen. 2025. Diffusion model-based image editing: A survey. *IEEE Transactions on Pattern Analysis and Machine Intelligence* (2025).
- [11] Thomas N Kipf and Max Welling. 2016. Semi-supervised classification with graph convolutional networks. *arXiv preprint arXiv:1609.02907* (2016).
- [12] Linghai Kong, Jiaming Cui, Haotian Sun, Yuchen Zhuang, B Aditya Prakash, and Chao Zhang. 2023. Autoregressive diffusion model for graph generation. In *International conference on machine learning*. PMLR, 17391–17408.
- [13] Srijan Kumar, Xikun Zhang, and Jure Leskovec. 2019. Predicting dynamic embedding trajectory in temporal interaction networks. In *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*. 1269–1278.
- [14] Timothée Lesort, Hugo Caselles-Dupré, Michael Garcia-Ortiz, Andrei Stoian, and David Filliat. 2019. Generative models from the perspective of continual learning. In *2019 International Joint Conference on Neural Networks (IJCNN)*. IEEE, 1–8.
- [15] Ao Li, Zhou Qin, Runshi Liu, Yiqun Yang, and Dong Li. 2019. Spam review detection with graph convolutional networks. In *Proceedings of the 28th ACM International conference on information and knowledge management*. 2703–2711.
- [16] Hanjie Li, Changsheng Li, Kaituo Feng, Ye Yuan, Guoren Wang, and Hongyuan Zha. 2024. Robust knowledge adaptation for dynamic graph neural networks. *IEEE Transactions on Knowledge and Data Engineering* (2024).
- [17] Yang Liu, Xiang Ao, Zidi Qin, Jianfeng Chi, Jinghua Feng, Hao Yang, and Qing He. 2021. Pick and choose: a GNN-based imbalanced learning approach for fraud detection. In *Proceedings of the web conference 2021*. 3168–3177.
- [18] Yixin Liu, Shirui Pan, Yu Guang Wang, Fei Xiong, Liang Wang, Qingfeng Chen, and Vincent CS Lee. 2023. Anomaly detection in dynamic graphs via transformer. *IEEE Transactions on Knowledge and Data Engineering* 35, 12 (2023), 12081–12094.
- [19] Yilun Liu, Ruihong Qiu, Yanran Tang, Hongzhi Yin, and Zi Huang. 2024. PUMA: Efficient Continual Graph Learning for Node Classification With Graph Condensation. *IEEE Transactions on Knowledge and Data Engineering* (2024).
- [20] Andrea Maracani, Umberto Micheli, Marco Toldo, and Pietro Zanuttigh. 2021. Recall: Replay-based continual learning in semantic segmentation. In *Proceedings of the IEEE/CVF international conference on computer vision*. 7026–7035.
- [21] Rui Ou, Kun Zhu, Nana Zhang, MengChu Zhou, and Changjun Jiang. 2025. CamFD: Semi-Supervised Camouflage-Aware Fraud Detection Based on Dynamic Graphs. *IEEE Transactions on Systems, Man, and Cybernetics: Systems* (2025), 1–15. doi:10.1109/TSMC.2025.3648571
- [22] David Rolnick, Arun Ahuja, Jonathan Schwarz, Timothy Lillicrap, and Gregory Wayne. 2019. Experience replay for continual learning. *Advances in neural information processing systems* 32 (2019).
- [23] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. 2015. U-net: Convolutional networks for biomedical image segmentation. In *Medical image computing and computer-assisted intervention—MICCAI 2015: 18th international conference, Munich, Germany, October 5–9, 2015, proceedings, part III* 18. Springer, 234–241.
- [24] Emanuele Rossi, Ben Chamberlain, Fabrizio Frasca, Davide Eynard, Federico Monti, and Michael Bronstein. 2020. Temporal Graph Networks for Deep Learning on Dynamic Graphs. In *ICML 2020 Workshop on Graph Representation Learning*.
- [25] Hanul Shin, Jung Kwon Lee, Jaehong Kim, and Jiwon Kim. 2017. Continual learning with deep generative replay. *Advances in neural information processing systems* 30 (2017).
- [26] Junwei Su, Difan Zou, Zijun Zhang, and Chuan Wu. 2023. Towards robust graph incremental learning on evolving graphs. In *International Conference on Machine Learning*. PMLR, 32728–32748.
- [27] Sheng Tian, Jihai Dong, Jintang Li, Wenlong Zhao, Xiaolong Xu, Bowen Song, Changhua Meng, Tianyi Zhang, Liang Chen, et al. 2023. Sad: Semi-supervised anomaly detection on dynamic graphs. *arXiv preprint arXiv:2305.13573* (2023).
- [28] Zongui Tian, Du Zhang, and Hong-Ning Dai. 2024. Continual Learning on Graphs: A Survey. *arXiv preprint arXiv:2402.06330* (2024).
- [29] Laurens Van der Maaten and Geoffrey Hinton. 2008. Visualizing data using t-SNE. *Journal of machine learning research* 9, 11 (2008).
- [30] A Vaswani. 2017. Attention is all you need. *Advances in Neural Information Processing Systems* (2017).
- [31] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Lio, and Yoshua Bengio. 2017. Graph attention networks. *arXiv preprint arXiv:1710.10903* (2017).
- [32] Clement Vignac, Igor Krawczuk, Antoine Siraudin, Bohan Wang, Volkan Cevher, and Pascal Frossard. 2022. Digress: Discrete denoising diffusion for graph generation. *arXiv preprint arXiv:2209.14734* (2022).
- [33] Cheng Wang, Hao Tang, Hangyu Zhu, and Changjun Jiang. 2023. Collaborative Prediction in Anti-Fraud System Over Multiple Credit Loan Platforms. *IEEE Transactions on Dependable and Secure Computing* 21, 4 (2023), 3580–3596.
- [34] Cheng Wang, Hao Tang, Hangyu Zhu, and Changjun Jiang. 2023. Collaborative Prediction in Anti-Fraud System Over Multiple Credit Loan Platforms. *IEEE Transactions on Dependable and Secure Computing* (2023).
- [35] Daixin Wang, Jianbin Lin, Peng Cui, Qianhui Jia, Zhen Wang, Yanming Fang, Quan Yu, Jun Zhou, Shuang Yang, and Yuan Qi. 2019. A semi-supervised graph attentive network for financial fraud detection. In *2019 IEEE International Conference on Data Mining (ICDM)*. IEEE, 598–607.
- [36] Junshan Wang, Wenhao Zhu, Guojie Song, and Liang Wang. 2022. Streaming graph neural networks with generative replay. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 1878–1888.
- [37] Xuhong Wang, Ding Lyu, Mengjian Li, Yang Xia, Qi Yang, Xinwen Wang, Xinguang Wang, Ping Cui, Yupu Yang, Bowen Sun, et al. 2021. Apan: Asynchronous propagation attention network for real-time temporal graph embedding. In *Proceedings of the 2021 international conference on management of data*. 2628–2638.
- [38] Yuchen Wang, Jinghui Zhang, Zhengjie Huang, Weibin Li, Shikun Feng, Ziheng Ma, Yu Sun, Dianhai Yu, Fang Dong, Jiahui Jin, et al. 2023. Label information enhanced fraud detection against low homophily in graphs. In *Proceedings of the ACM Web Conference 2023*. 406–416.
- [39] Sheng Xiang, Guibin Zhang, Dawei Cheng, and Ying Zhang. 2025. Enhancing Attribute-Driven Fraud Detection With Risk-Aware Graph Representation. *IEEE Transactions on Knowledge and Data Engineering* (2025).
- [40] Sheng Xiang, Mingzhi Zhu, Dawei Cheng, Enxia Li, Ruihui Zhao, Yi Ouyang, Ling Chen, and Yefeng Zheng. 2023. Semi-supervised credit card fraud detection via attribute-driven graph representation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 37. 14557–14565.
- [41] Da Xu, Chuanwei Ruan, Evren Korpeoglu, Sushant Kumar, and Kannan Achan. 2020. Inductive representation learning on temporal graphs. *arXiv preprint arXiv:2002.07962* (2020).
- [42] Minkai Xu, Tomas Geffner, Karsten Kreis, Weili Nie, Yilun Xu, Jure Leskovec, Stefano Ermon, and Arash Vahdat. 2024. Energy-based diffusion language models for text generation. *arXiv preprint arXiv:2410.21357* (2024).
- [43] Jinlong Xue, Yayue Deng, Yingming Gao, and Ya Li. 2024. Auffusion: Leveraging the power of diffusion and large language models for text-to-audio generation. *IEEE/ACM Transactions on Audio, Speech, and Language Processing* (2024).
- [44] Jiasheng Ye, Zaixiang Zheng, Yu Bao, Lihua Qian, and Quanquan Gu. 2023. Diffusion language models can perform many tasks with scaling and instruction-finetuning. *arXiv preprint arXiv:2308.12219* (2023).
- [45] Jianke Yu, Hanchen Wang, Xiaoyang Wang, Zhao Li, Lu Qin, Wenjie Zhang, Jian Liao, Ying Zhang, and Bailin Yang. 2024. Temporal Insights for Group-Based Fraud Detection on e-Commerce Platforms. *IEEE Transactions on Knowledge and Data Engineering* (2024).
- [46] Qi Yuan, Yang Liu, Yateng Tang, Xinhuan Chen, Xuehao Zheng, Qing He, and Xiang Ao. 2025. Dynamic Graph Learning with Static Relations for Credit Risk Assessment. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 13133–13141.
- [47] Zirui Yuan, Minglai Shao, and Qiben Yan. 2023. Motif-level Anomaly Detection in Dynamic Graphs. *IEEE Transactions on Information Forensics and Security*

- (2023).
- [48] Si Zhang, Hanghang Tong, Jiejun Xu, and Ross Maciejewski. 2019. Graph convolutional networks: a comprehensive review. *Computational Social Networks* 6, 1 (2019), 1–23.
 - [49] Xikun Zhang, Dongjin Song, and Dacheng Tao. 2024. Continual learning on graphs: Challenges, solutions, and opportunities. *arXiv preprint arXiv:2402.11565* (2024).
 - [50] Fan Zhou and Chengtai Cao. 2021. Overcoming catastrophic forgetting in graph neural networks with experience replay. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 35. 4714–4722.
 - [51] Xiaoqian Zhu, Xiang Ao, Zidi Qin, Yanpeng Chang, Yang Liu, Qing He, and Jianping Li. 2021. Intelligent financial fraud detection practices in post-pandemic era. *The Innovation* 2, 4 (2021).
 - [52] Yao Zou and Dawei Cheng. 2025. Effective high-order graph representation learning for credit card fraud detection. *arXiv preprint arXiv:2503.01556* (2025).

A Justification of Proposition 1

We now provide a justification for Proposition 1, showing that the dependencies among multiple input matrices can be captured entirely in the reverse process, and the forward noising process can be fully factorized.

Let the input data $\mathbf{X}_0$ consist of K non-independent matrices:

$$\mathbf{X}_0 = (\mathbf{X}_0^1, \mathbf{X}_0^2, \dots, \mathbf{X}_0^K), \quad (11)$$

where the joint distribution $p_{\text{data}}(\mathbf{X}_0)$ exhibits unknown dependencies across matrices $\mathbf{X}_0^k$.

With a forward noising kernel $q(\mathbf{X}_s | \mathbf{X}_{s-1})$, the forward noising process can be formulated as:

$$q(\mathbf{X}_{1:S} | \mathbf{X}_0) = \prod_{s=1}^S q(\mathbf{X}_s | \mathbf{X}_{s-1}), \quad (12)$$

where S is the number of diffusion steps.

The reverse denoising process gradually recovers the clean data with a learnable denoising kernel $p_\theta(\mathbf{X}_{s-1} | \mathbf{X}_s)$:

$$p_\theta(\mathbf{X}_{0:S}) = p(\mathbf{X}_S) \prod_{s=1}^S p_\theta(\mathbf{X}_{s-1} | \mathbf{X}_s), \quad (13)$$

where $p(\mathbf{X}_S)$ is a standard Gaussian.

The diffusion model is trained by maximizing a variational lower bound (ELBO) on the marginal log-likelihood of the observed data, which equivalently minimizes its negative log-likelihood (NLL):

$$\begin{aligned} \mathbb{E}_q [-\log p_\theta(\mathbf{X}_0)] &\leq \mathbb{E}_q \left[-\log \frac{p_\theta(\mathbf{X}_0, \mathbf{X}_{1:S})}{q(\mathbf{X}_{1:S} | \mathbf{X}_0)} \right] \\ &= \mathbb{E}_q \left[-\log p(\mathbf{X}_S) - \sum_{s=1}^S \log \frac{p_\theta(\mathbf{X}_{s-1} | \mathbf{X}_s)}{q(\mathbf{X}_s | \mathbf{X}_{s-1})} \right]. \end{aligned} \quad (14)$$

The variational lower bound formulation above can be further decomposed into interpretable and tractable components. By rearranging the terms and isolating the KL divergences and likelihood contribution, the ELBO can be expressed as:

$$\begin{aligned} \mathcal{L}_{\text{ELBO}} &= \underbrace{-\text{KL}(q(\mathbf{X}_S | \mathbf{X}_0) \| p(\mathbf{X}_S))}_{\text{prior loss}} \\ &\quad - \underbrace{\sum_{s=2}^S \mathbb{E}_{q(\mathbf{X}_{s:S} | \mathbf{X}_0)} [\text{KL}(q(\mathbf{X}_{s-1} | \mathbf{X}_s, \mathbf{X}_0) \| p_\theta(\mathbf{X}_{s-1} | \mathbf{X}_s))]}_{\text{denoising KL loss}} \\ &\quad + \underbrace{\mathbb{E}_{q(\mathbf{X}_1 | \mathbf{X}_0)} [\log p_\theta(\mathbf{X}_0 | \mathbf{X}_1)]}_{\text{reconstruction likelihood}}. \end{aligned} \quad (15)$$

From Equation (15), several key observations can be made. First, the prior loss term is independent of the learnable parameters and thus remains constant throughout training. Second, the forward process $q(\cdot)$ only appears within the KL divergence, serving as a sampling mechanism. Third, the modeling capacity of diffusion models is determined by the parameterized reverse process $p_\theta(\cdot)$.

This formulation highlights that the choice of forward process is not unique as long as it enables efficient sampling and leads to a tractable ELBO. In practice, we adopt a factorized Gaussian forward process as defined in Equations (4)(5), which substantially simplifies the forward process while allowing the reverse process to capture complex dependencies among input data.

B Dataset Information

In this work, we conduct extensive experiments on four real-world streaming fraud detection datasets.

- (1) **S-FFSD** is a simulated financial transaction fraud dataset, where nodes represent users and edges denote transactions. A positive labeled edge indicates a fraudulent transaction.
- (2) **Credit** is a private credit card fraud dataset collected from a major commercial bank in China, containing over 500K credit card transactions. The ground truth labels are confirmed by domain experts.
- (3) **Wikipedia** is a web edit network, where nodes represent users and web pages, and edges denote edit interactions. A positive labeled edge indicates that a fraudulent user edits a web page.
- (4) **Reddit** is a posting network, where node represent users and subreddit, and edges denote posting interactions. A positive labeled edge indicates that a fraudulent user posts on a subreddit.

C Baseline Description

We compare STG-DGR with two conventional machine learning methods in fraud detection (Random Forest (RF) and Support Vector Machine (SVM)), as well as seventeen state-of-the-art graph learning baselines. Brief descriptions of graph learning baselines are listed as follows.

- **SemiGNN (2019)** [35] is a graph fraud detection model with a hierarchical attention structure, which can capture node- and view-level information to fully exploit labeled samples.
- **CARE (2020)** [3] is a graph fraud detection model that aims at detecting feature-level and relation-level camouflage with a label-aware neighbor selector.
- **TGAT (2019)** [41] is a robust DGNN model that extends attention mechanism to temporal graph encoding. It has been widely applied in various temporal graph learning fields.
- **TADDY (2023)** [18] integrates spatial encoding and temporal encoding schemes to capture the coupled spatial-temporal patterns in temporal graphs.
- **MADG (2023)** [47] proposes a motif-augmented GCN model and a temporal self-attention module to learn motif-level representations on temporal graphs.
- **GTAN (2023)** [40] designs a gated temporal graph attention network and a risk propagation mechanism to learn the multi-hop risk information in temporal graphs.

Table 4: Overview of STG-DGR and compared baselines.

Methods	Graph	Temporal	Continual
RF, SVM	×	×	×
SemiGNN, CARE	⊠	×	×
TGAT, TADDY, MADG, GTAN, SAD, AdaDyGNN	⊠	⊠	×
ER-GNN, SSRM, SGNN-GR, DSLR	⊠	×	⊠
ER-GNN+TGAT, SSRM+TGAT, SGNN-GR+TGAT, DSLR+TGAT	⊠	⊠	⊠
STG-DGR	⊠	⊠	⊠

- **SAD (2023)** [27] leverages Gaussian distribution to model the fraud scores of unlabeled and benign samples on temporal graphs.
- **AdaDyGNN (2024)** [16] explores robust knowledge adaptation by employing reinforcement learning to adaptively update the embedding on temporal graphs.
- **RG TAN (2025)** [39] further enhances the risk propagation strategy in GTAN and models the fraud patterns among transactions.
- **ER-GNN (2021)** [50] addresses catastrophic forgetting in streaming graph learning by storing and revisiting historical graph samples.
- **SSRM (2023)** [26] focuses on the structural shift issue in graph continual learning tasks and designs a regularization-based methods to mitigate forgetting problem caused by this issue.
- **SGNN-GR (2022)** [36] is a generative replay model that employs a GAN model to generate random walk sequences of a target node to approximate its heterogeneous neighborhood.
- **DSLR (2024)** [2] is a state-of-the-art sampling replay (SR) method that proposes a coverage-based diversity approach to enhance the diversity of replay samples.
- **ER-GNN+TGAT (2021)** is the temporal version of ER-GNN. We replace the static GNN backbone of ER-GNN model with TGAT, the same backbone of proposed STG-DGR.
- **SGNN-GR+TGAT (2022)** is the temporal version of SGNN-GR.
- **SSRM+TGAT (2023)** is the temporal version of SSRM.
- **DSLR+TGAT (2024)** is the temporal version of DSLR.

As shown in Table 4, the compared baseline can be categorized into five groups. (1) **Traditional machine learning models.** RF and SVM; (2) **Static GNN models.** SemiGNN and CARE; (3) **Temporal GNN models.** TGAT, TADDY, MADG, GTAN, SAD, AdaDyGNN, and RG TAN; (4) **Static graph continual learning (GCL) models.** ER-GNN, SSRM, SGNN-GR, and DSLR; (5) **Temporal GCL models.** ER-GNN+TGAT, SSRM+TGAT, SGNN-GR+TGAT, and DSLR+TGAT.

D Algorithm Overview

Algorithm 1 outlines the continual training procedure of STG-DGR for a new task i , which incrementally updates the model parameters in previous task. The algorithm is well-aligned with the four stages in Figure 4: 1) CSP process the raw samples in the new task into hierarchical subgraphs (line 1); 2) DSG is trained with the processed new samples by forward noising and reverse denoising (lines 2-5); 3) DSG generates replay samples by denoising from random graphs (line 6); 4) Train TGNN with the joint of new and replay samples (lines 7-8).

Algorithm 1 Training Procedure of STG-DGR for Task i .

Input: Task i ΔG_i ; Ground-truth labels of new samples $\mathbf{y}_{\text{new}}$;

Model parameters in previous task $\theta_{i-1}^{\text{CSP}}, \theta_{i-1}^{\text{DSG}}, \theta_{i-1}^{\text{DGNN}}$

Output: Updated model parameters $\theta_i^{\text{CSP}}, \theta_i^{\text{DSG}}, \theta_i^{\text{DGNN}}$

- 1: $\mathbf{g}_{\text{new}} = (\mathbf{m}^0, \dots, \mathbf{m}^L) = \text{CSP}(\Delta G_i)$; // processed subgraphs
- 2: $s \sim \text{Uniform}(1, \dots, S)$, $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$;
- 3: $\mathbf{m}_s^l = \sqrt{\alpha_s} \mathbf{m}_0^l + \sqrt{1 - \alpha_s} \epsilon$, $l = 0, \dots, L$; // forward diffusion
- 4: Predict noise $\epsilon_\theta(\mathbf{m}_s^l, s)$ with THDN; // backward denoising
- 5: $\mathcal{L}_{\text{dif}} = \sum_{l=0}^L \mathbb{E} [\|\epsilon - \epsilon_\theta(\mathbf{m}_s^l, s)\|^2]$; // diffusion loss
- 6: Generate replay samples $\mathbf{g}_{\text{replay}}, \mathbf{y}_{\text{replay}}$ by denoising from random graphs;
- 7: $\mathbf{s}_{\text{new}}, \mathbf{s}_{\text{replay}} = \text{DGNN}(\mathbf{g}_{\text{new}}, \mathbf{g}_{\text{replay}})$; // fraud score
- 8: $\mathcal{L}_{\text{cla}} = \text{BCE}(\mathbf{s}_{\text{new}}, \mathbf{y}_{\text{new}}) + \text{BCE}(\mathbf{s}_{\text{replay}}, \mathbf{y}_{\text{replay}})$;
// classification loss
- 9: $\theta_i^{\text{CSP}} = \theta_{i-1}^{\text{CSP}} - \eta \nabla \mathcal{L}_{\text{cla}}$;
- 10: $\theta_i^{\text{DSG}} = \theta_{i-1}^{\text{DSG}} - \eta \nabla \mathcal{L}_{\text{dif}}$;
- 11: $\theta_i^{\text{DGNN}} = \theta_{i-1}^{\text{DGNN}} - \eta \nabla \mathcal{L}_{\text{cla}}$;
- 12: **return** $\theta_i^{\text{CSP}}, \theta_i^{\text{DSG}}, \theta_i^{\text{DGNN}}$;

E Additional Evaluation on Long-term Forgetting

As a complement to Figure 6 in Section 5.3, we further evaluate the models' long-term performance on Wikipedia and Reddit datasets. As shown in figures 6 and 10, STG-DGR demonstrates consistent superiority in mitigating catastrophic forgetting of previous learned patterns across four streaming datasets through diffusion-based generative replay.

F Memory and Time Efficiency

In this section, we conduct a comprehensive evaluation of the memory and time efficiency of STG-DGR compared to existing graph continual learning (GCL) methods. The methods can be categorized into two groups: (1) *Sampling Replay methods (SRs)*, including ER-GNN [50], SSRM [26] and DSLR [2] (2) *Generative Replay methods (GRs)*, represented by SGNN-GR [36] as well as our proposed STG-DGR. As the main differences among the three SR methods lie in their sample selection strategies, they do not exhibit significant differences in memory and time efficiency. Therefore, we refer to them collectively as SRs for ease of discussion and report representative experimental results using ER-GNN [50].

Memory Efficiency. SRs store historical graph samples for replay, resulting in the memory complexity of $O(TSM^L d_{\text{raw}})$, where T is the number of tasks, S is the number of stored samples per task, L is the number of GNN layers, M is the number of neighbors per layer, and d_{raw} is the raw feature dimension. While GRs train replay generators to produce replay samples, thus the memory complexity is determined by the size of replay generators (the DSG module in STG-DGR). SGNN-GR employs a GAN-based replay generator, whose memory complexity is $O(Dd^2)$, where D is model depth and d is the hidden feature dimension. DSG in STG-DGR has a memory complexity of $O((L+1)Bd^2)$, where L is the

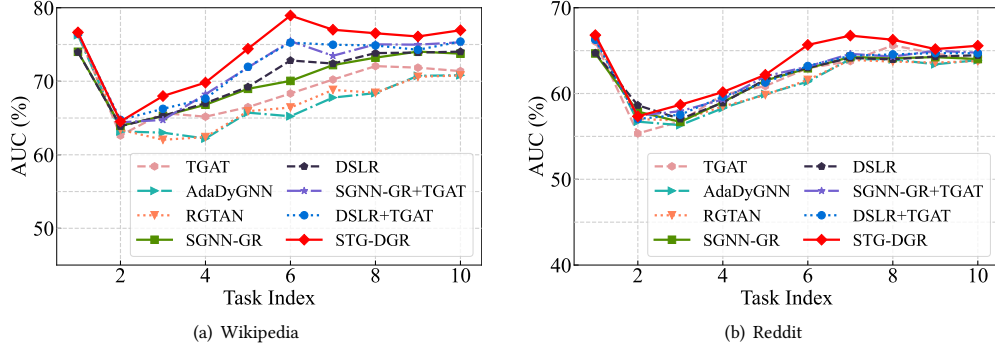


Figure 10: Additional evaluation on long-term forgetting for Wikipedia and Reddit datasets.

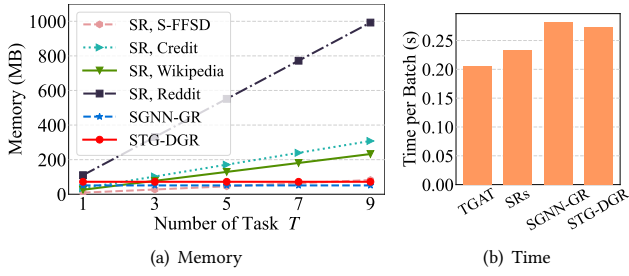


Figure 11: Evaluation on memory and time efficiency.

number of GNN layers and B is the number of transformer blocks, which is comparable to SGNN-GR. Figure 11(a) shows the additional memory consumption introduced by the replay mechanisms. For SRs, we set the ratio of stored samples to 5% [50]. The memory consumption of SRs increases with the raw feature dimension and dataset size, making them less suitable for large-scale tasks. In contrast, as GRs, STG-DGR and SGNN-GR exhibit consistent memory consumption across different datasets and task scales, since the size of the replay generator depends only on the model architecture. Although STG-DGR consumes slightly more memory than SGNN-GR, it offers better parallelism and training-time efficiency.

Offline Training Time Efficiency. Both SRs and GRs require training the GNN backbone, whose time complexity is approximately $O(NM^Ld^2)$, where N denotes the number of training samples, including both new and replay samples $N = N_{\text{new}} + N_{\text{replay}}$. As shown in Figure 2(a), SRs must maintain stored samples as STG evolves, introducing an additional time complexity of $O(TSM^Ld_{\text{raw}})$. In contrast, GRs do not store samples but incur extra training time for the replay generator. Specifically, the time complexity for generator training is $O(N_{\text{new}}Rd^2)$ for SGNN-GR and $O(N_{\text{new}}(L+1)Bd^2)$ for STG-DGR, where R is the length of random walk sequences in SGNN-GR. In general, the time complexity of offline training for GRs is higher than that for SRs, as GRs trade computational time for improved scalability and performance. However, the additional cost of STG-DGR $O(N_{\text{new}}(L+1)Bd^2)$ is reasonable relative to backbone training $O(NM^Ld^2)$, since $(L+1)B \ll M^L$ and $N_{\text{new}} < N$. As shown in Figure 11(b), STG-DGR is slightly slower than the backbone TGAT and SR methods, and is faster than SGNN-GR due to better parallelism of DSG’s transformer-based architecture.

Online Inference Time Efficiency. Although training a replay generator inevitably slows down the offline training of STG-DGR, **it does not affect the online inference speed**. This is because the replay generator is used only during offline training to better optimize the GNN backbone, while online inference involves only the GNN forward process.